

Lab 7: การจัดการทรัพยากรด้วย Tagging (Managing Resources with Tagging)

แปลและเรียบเรียงจากไฟล์ต้นฉบับ lab7.md · ครบทั้ง 6 ส่วน

ส่วน 1 / 6

โมดูล 10: Lab 7 – การจัดการทรัพยากรด้วย Tagging (Managing Resources with Tagging)

(ต้นฉบับ .md มีคำว่า "choosing" หลงอยู่บรรทัดแรกก่อนหัวข้อ ซึ่งไม่ใช่เนื้อหา)

แล็บนี้แบ่งเป็นสองส่วน:

- ในส่วนงาน (Task) ของแล็บ คุณจะใช้ AWS Command Line Interface (CLI) ตรวจสอบ tags ที่กำหนดให้ Amazon EC2 instances จำนวนหนึ่ง จากนั้นใช้สคริปต์ที่เตรียมไว้ให้ ปิดและเปิด Amazon EC2 instances หลายตัวพร้อมกันตาม tags
- ในส่วนโจทย์ท้าทาย (Challenge) คุณจะถูกท้าทายให้คิดวิธี terminate instances ที่ไม่ได้ติด tags ตามที่กำหนด

วัตถุประสงค์ (Objectives)

หลังจากทำแล็บนี้เสร็จ คุณจะสามารถ:

- ติด tags ให้ทรัพยากร AWS ที่มีอยู่
- ค้นหาทรัพยากรตาม tags
- ใช้ AWS CLI หรือ AWS SDK for PHP หยุดและ terminate Amazon EC2 instances ตามคุณลักษณะบางอย่างของทรัพยากร

ระยะเวลา (Duration)

แล็บนี้ใช้เวลาประมาณ 30 นาที

สถานการณ์ (Scenario)

สภาพแวดล้อมของแล็บนี้ (ดังภาพด้านล่าง) ประกอบด้วย:

- Amazon VPC ชื่อ Lab VPC
- Public subnet
- Private subnet
- Amazon EC2 Linux instance ชื่อ CommandHost [เครื่องมือ AWS Command Line Interface (CLI) ถูกติดตั้งและตั้งค่าไว้ให้แล้วบน instance นี้]
- Amazon EC2 Linux instances 8 ตัว
- private instances มี tags แบบกำหนดเองสามตัว:

[ภาพตาราง custom tags (lab-7-md-table.png) – เนื้อหาดังกล่าวมาจากส่วนที่ถูกคอมเมนต์ไว้ในไฟล์ .md ต้นฉบับ:]

ชื่อ Tag	เนื้อหา
Project	โปรเจกต์ที่ instance สังกัด instances ในแล็บนี้สังกัดหนึ่งในสองโปรเจกต์: ERPSys และ Experiment1
Version	เวอร์ชันของโปรเจกต์ที่ instance สังกัด ขณะนี้ tag Version ทั้งหมดตั้งเป็น 1.0
Environment	คำใดคำหนึ่งในสามคำ: development , staging หรือ production

ในส่วนงานของแล็บ คุณจะล็อกอินเข้า Command Host และรันคำสั่งเพื่อค้นหาและเปลี่ยน tag Version บน instances ที่เป็น development ทั้งหมด จากนั้นตรวจสอบว่าหลายตัวที่แสดงวิธีใช้ไวยากรณ์ JMESPath ที่ตัวเลือก --query ของ AWS CLI รองรับ เพื่อให้ได้ผลลัพธ์ที่จัดรูปแบบอย่างดี จากนั้นจะใช้สคริปต์ที่เตรียมไว้ให้หยุดและเริ่ม instances ทั้งหมดที่ติด tag ว่าสังกัดสภาพแวดล้อม development ใหม่

[ภาพสถาปัตยกรรมทรัพยากร (ResourcesArchitecture)]

ส่วน 2 / 6

การเข้าใช้ AWS Management Console

- 1. ที่ด้านบนของคำแนะนำนี้ เลือก **Start Lab** เพื่อเริ่มแล็บ
 - แผง Start Lab จะเปิดขึ้นและแสดงสถานะของแล็บ
- 2. รอจนเห็นข้อความ **"Lab status: ready"** แล้วเลือก **X** เพื่อปิดแผง Start Lab
- 3. ที่ด้านบนของคำแนะนำนี้ เลือก **AWS**
 - จะเปิด AWS Management Console ในแท็บเบราว์เซอร์ใหม่ ระบบจะล็อกอินให้อัตโนมัติ
 - เคล็ดลับ (Tip):** หากแท็บใหม่ไม่เปิดขึ้น โดยทั่วไปจะมีแบนเนอร์หรือไอคอนที่ด้านบนของเบราว์เซอร์แจ้งว่าเบราว์เซอร์กำลังบล็อกป๊อปอัพ ให้เลือกแบนเนอร์หรือไอคอนนั้นแล้วเลือก "Allow pop ups"

- 4. จัดแท็บ AWS Management Console ให้แสดงคู่กับคำแนะนำนี้ ทางที่ดีที่สุดคือให้เห็นทั้งสองแท็บพร้อมกัน
 - คำเตือน: กรุณาอย่าเปลี่ยน Region ระหว่างทำแล็บนี้

ส่วน 3 / 6

งานที่ 1: ใช้ Tags จัดการทรัพยากร

ในงานนี้ คุณจะล็อกอินเข้า Command Host และใช้ AWS CLI ค้นหาชุดทรัพยากรตาม tags จากนั้นใช้ AWS CLI เปลี่ยนค่าของ tag ตัวหนึ่ง

เชื่อมต่อกับ Command Host

คำแนะนำต่อไปนี้จะต่างกันเล็กน้อย ขึ้นอยู่กับว่าคุณใช้ Windows หรือ Mac/Linux

ผู้ใช้ Windows: ใช้ SSH เชื่อมต่อ

หมายเหตุ: คำแนะนำนี้สำหรับผู้ที่ใช้ Windows เท่านั้น หาก你用 macOS หรือ Linux ให้ข้ามไปยังส่วนถัดไป

- 5. ใน **AWS Management Console** ที่เมนู **Services** เลือก **EC2**
- 6. ในแถบนำทางด้านซ้าย เลือก **Instances**
- 7. เลือก **Command Host**
- 8. คัดลอก **IPv4 Public IP** จาก Description ในแผงด้านล่าง
- 9. อ่านสามหัวข้อย่อยในขั้นนี้ให้จบก่อนเริ่มทำ เพราะคุณจะไม่เห็นคำแนะนำนี้ขณะแผง Details เปิดอยู่
 - เลือกเมนู dropdown **Details** เหนือคำแนะนำที่คุณกำลังอ่าน แล้วเลือก **Show** หน้าต่าง Credentials จะเปิดขึ้น
 - เลือกปุ่ม **Download PPK** และบันทึกไฟล์ **labsuser.ppk** โดยทั่วไปเบราว์เซอร์จะบันทึกไว้ในไดเรกทอรี Downloads
 - จากนั้นออกจากแผง Details โดยเลือก **X**
- 10. ดาวน์โหลดซอฟต์แวร์ที่จำเป็น
 - คุณจะใช้ **PuTTY** เพื่อ SSH เข้า Amazon EC2 instances หากยังไม่มี PuTTY ให้ดาวน์โหลดที่ <https://the.earth.li/~sgtatham/putty/latest/w64/putty.exe>
- 11. เปิด **putty.exe**
- 12. ตั้งค่า PuTTY ไม่ให้หมดเวลา:
 - เลือก **Connection**
 - ตั้ง **Seconds between keepalives** เป็น 30
 - ทำให้คุณเปิด session ของ PuTTY ไว้ได้นานขึ้น
- 13. ตั้งค่า session ของ PuTTY:
 - เลือก **Session**
 - Host Name (or IP address):** วาง *Public DNS หรือ IPv4 address* ของ Bastion Host instance ที่จดไว้ก่อนหน้านี้ (ในแล็บนี้คือ Command Host)
 - กลับไป PuTTY ในรายการ **Connection** ขยาย **SSH**
 - เลือก **Auth** แล้วขยาย **Credentials**
 - ได้ **Private key file for authentication:** เลือก **Browse**
 - เลือกไฟล์ ***labsuser.ppk*** ที่ดาวน์โหลดไว้ แล้วเลือก **Open**
 - เลือก **Open** อีกครั้ง
- 14. เพื่อเชื่อมต่อและเชื่อมต่อกับโฮสต์ เลือก **Accept**
- 15. เมื่อถูกถาม **login as** ให้ใส่: **ec2-user**
 - จะเชื่อมต่อคุณเข้ากับ EC2 instance
- 16. ผู้ใช้ Windows: ข้ามไปยังงานถัดไปได้เลย

ผู้ใช้ Mac และ Linux

คำแนะนำนี้สำหรับผู้ที่ใช้ Mac/Linux เท่านั้น หาก你用 Windows ให้ข้ามไปยังงานถัดไป

- 17. ใน **AWS Management Console** ที่เมนู **Services** เลือก **EC2**
- 18. ในแถบนำทางด้านซ้าย เลือก **Instances**
- 19. เลือก **Command Host**
- 20. คัดลอก **IPv4 Public IP** จาก Description ในแผงด้านล่าง
- 21. อ่านสามหัวข้อย่อยในขั้นนี้ให้จบก่อนเริ่มทำ เพราะคุณจะไม่เห็นคำแนะนำนี้ขณะแผง Details เปิดอยู่
 - เลือกเมนู dropdown **Details** เหนือคำแนะนำที่คุณกำลังอ่าน แล้วเลือก **Show** หน้าต่าง Credentials จะเปิดขึ้น
 - เลือกปุ่ม **Download PEM** และบันทึกไฟล์ **labsuser.pem**
 - จากนั้นออกจากแผง Details โดยเลือก **X**

- 22. เปิดหน้าต่าง terminal และเปลี่ยนไดเรกทอรี (cd) ไปยังไดเรกทอรีที่ดาวน์โหลดไฟล์ labsuser.pem ไว้ ตัวอย่างเช่น หากบันทึกไว้ในไดเรกทอรี Downloads ให้รันคำสั่ง:

```
cd ~/Downloads
```

- 23. เปลี่ยนสิทธิ์ของคีย์ให้อ่านได้อย่างเดียว ด้วยคำสั่ง:

```
chmod 400 labsuser.pem
```

- 24. กลับไปที่หน้าต่าง terminal และรันคำสั่งนี้ (แทน <public-ip> ด้วยค่า **Public IPv4** ที่คัดลอกไว้ก่อนหน้านี้):

```
ssh -i labsuser.pem ec2-user@<public-ip>
```

- 25. พิมพ์ yes เมื่อถูกถามว่าจะอนุญาตการเชื่อมต่อครั้งแรกกับ SSH server ระยะไกลหรือไม่
 - เนื่องจากคุณใช้ key pair ในการยืนยันตัวตน จะไม่มีการถามรหัสผ่าน

ค้นหา Development Instances ของโปรเจกต์

เมื่อล็อกอินแล้ว คุณใช้ AWS CLI ค้นหาทรัพยากรใน private subnet ที่สังกัดโปรเจกต์ **ERPSYSTEM** และอยู่ใน Environment ชื่อ **development** ได้ คุณจะเห็นวิธีใช้ตัวเลือก **--query** ของ AWS CLI สร้างผลลัพธ์ที่จัดรูปแบบอย่างดียิ่ง

- 26. เพื่อค้นหา instances ทั้งหมดในบัญชีที่ติด tag **Project** ด้วยค่า **ERPSYSTEM** ให้คัดลอกคำสั่งต่อไปนี้แล้วรันในหน้าต่าง Linux terminal:

```
aws ec2 describe-instances --filter "Name=tag:Project,Values=ERPSYSTEM"
```

- คำสั่งควรแสดงชุดพารามิเตอร์ทั้งหมดของ instances ทั้งเจ็ดตัวที่ติด tag **Project=ERPSYSTEM** ผลลัพธ์มีจำนวนมาก และส่วนใหญ่ไม่เกี่ยวกับแท็กนี้ ในขั้นถัดไป คุณจะใช้พารามิเตอร์ **--query** จำกัดผลลัพธ์ให้แคบลง
- 27. ใช้พารามิเตอร์ **--query** จำกัดผลลัพธ์ของคำสั่งก่อนหน้านี้ให้เหลือเฉพาะ instance ID ของ instance ที่พบ:

```
aws ec2 describe-instances --filter "Name=tag:Project,Values=ERPSYSTEM" --query 'Reservations[*].Instances[*].InstanceId'
```

- ผลลัพธ์จะเป็นรายการ instance IDs:

```
[
  [
    "i-135b491e"
  ],
  [
    "i-3e584a33"
  ],
  ...
]
```

- คำสั่ง **--query** ในตัวอย่างนี้ใช้ไวยากรณ์ wildcard ของ JMESPath ระบุว่าคำสั่งควรวนผ่านทุก reservations และทุก instances แล้วส่ง InstanceId ของแต่ละ instance ในผลลัพธ์กลับมา
- ดึงว่าการส่งคุณสมบัติทุกตัวของ instances กลับมา แต่ถ้าคุณต้องการหลายฟิลด์ในผลลัพธ์?
- 28. คัดลอกคำสั่งต่อไปนี้แล้วรันในหน้าต่าง Linux terminal เพื่อให้ผลลัพธ์มีทั้ง instance ID และ Availability Zone ของแต่ละ instance:

```
aws ec2 describe-instances --filter "Name=tag:Project,Values=ERPSYSTEM" --query 'Reservations[*].Instances[*].{ID:InstanceId,AZ:Placement.AvailabilityZone}'
```

- แต่ละผลลัพธ์จะส่งชื่อ/ค่ากลับมาสองคู่
- คำสั่งนี้ต่อด้วยการใช้ไวยากรณ์ JMESPath ของคำสั่งก่อนหน้านี้ โดยใช้วงเล็บปีกการะบุ query หลายคุณสมบัติของแต่ละ instance ที่ส่งกลับมา: `object.{Alias1:PropertyName1, Alias2:PropertyName2, [...]}`
- ดังที่เห็น คุณระบุชื่อแทน (alias) ของแต่ละคุณสมบัติได้ เพื่อให้ผลลัพธ์กระชับขึ้น
- จากผลลัพธ์นี้ คุณเห็นชัดว่าตัวกรองทำงาน และเห็นเฉพาะ instances ที่สังกัดโปรเจกต์ **ERPSYSTEM** อย่างไรก็ตาม คุณอาจยังระบุไม่ได้ว่า instances ใดถูกส่งกลับมาจากข้อมูลนี้ ในขั้นถัดไป คุณจะเห็นวิธีใส่ค่าของ tags แบบกำหนดเองในผลลัพธ์
- 29. เพื่อใส่ค่าของ tag **Project** ในผลลัพธ์ ให้คัดลอกและรันคำสั่งต่อไปนี้ใน Linux terminal:

```
aws ec2 describe-instances --filter "Name=tag:Project,Values=ERPSYSTEM" --query 'Reservations[*].Instances[*].{ID:InstanceId,AZ:Placement.AvailabilityZone,Project:Tags[?Key==`Project`] | [0].Value}'
```

- ตอนนี้ผลลัพธ์มีค่าของ tag Project:

```
[[{
  "Project": "ERPSYSTEM",
  "AZ": "us-west-2a",
  "ID": "i-3250b838"
}], ...]
```

- ค่าของ tag ที่ระบุชื่อได้ผ่าน JMESPath query โดยใช้ไวยากรณ์ต่อไปนี้:

```
Tags[?Key==`Project`] | [0].Value
```

- ไวยากรณ์นี้สั่งให้ JMESPath หาทุกองค์ประกอบในอาร์เรย์ **Tags** ที่มีค่า **Key** เป็น **Project** ผลลัพธ์ของคำสั่งนั้น (ซึ่งเป็นองค์ประกอบ Tags ตัวเดียว) ถูกส่งต่อ (pipe) ไปยังอีกคำสั่งหนึ่งที่เลือกองค์ประกอบแรกของชุดที่กรองแล้ว และเลือกพารามิเตอร์ชื่อ **Value** ซึ่งคือค่าของ tag **Project** จากนั้นผลลัพธ์นี้ถูกกำหนดชื่อแทนว่า **Project**
- 30. คัดลอกและรันคำสั่งต่อไปนี้เพื่อใส่ tags Environment และ Version ในผลลัพธ์ด้วย:

```
aws ec2 describe-instances --filter "Name=tag:Project,Values=ERPSYSTEM" --query 'Reservations[*].Instances[*].{ID:InstanceId,AZ:Placement.AvailabilityZone,Project:Tags[?Key==`Project`] | [0].Value,Environment:Tags[?Key==`Environment`] | [0].Value,Version:Tags[?Key==`Version`] | [0].Value}'
```

- ผลลัพธ์จะให้ภาพที่ครบถ้วนขึ้นของ instances ที่สังกัดโปรเจกต์ชื่อ **ERPSYSTEM** ในปัจจุบัน:

```
[[{
  "Environment": "production",
  "Project": "ERPSYSTEM",
  "Version": "1.0",
  "AZ": "us-west-2a",
  "ID": "i-3250b838"
}],
...
]
```

- 31. สุดท้าย เพิ่มตัวกรอง tag ตัวที่สองเพื่อดูเฉพาะ instances ที่สังกัดโปรเจกต์ **ERPSYSTEM** และอยู่ใน Environment ชื่อ **development**:

```
aws ec2 describe-instances --filter "Name=tag:Project,Values=ERPSYSTEM" "Name=tag:Environment,Values=development" --query 'Reservations[*].Instances[*].{ID:InstanceId,AZ:Placement.AvailabilityZone,Project:Tags[?Key==`Project`] | [0].Value,Environment:Tags[?Key==`Environment`] | [0].Value,Version:Tags[?Key==`Version`] | [0].Value}'
```

- คำสั่งนี้ควรส่ง instances กลับมาเพียงสองตัว ทั้งคู่มีค่า tag **Project** เป็น **ERPSYSTEM** และค่า tag **Environment** เป็น **development**:

```
[[{
  "Environment": "development",
  "Project": "ERPSYSTEM",
  "Version": "1.0",
  "AZ": "us-west-2a",
  "ID": "i-9552ba9f"
}],
...
]
```

เปลี่ยน Version Tag สำหรับกระบวนการ Development

ในขั้นตอนนี้ คุณจะเปลี่ยน tags **Version** ทั้งหมดบน instances ที่ระบุเป็น **development** ของโปรเจกต์ **ERPSYSTEM**

คุณอาจตั้งคำถามสมบัติเหล่านี้ที่ละ instance ได้ แต่วิธีอัตโนมัติใช้งานได้ง่ายกว่า คุณใช้ Linux Bash shell script ง่าย ๆ ต่อยอดจาก query ที่สร้างขึ้นก่อนหน้านี้ และแก้ไขรายการ tags เป็นชุด (batch) ได้

- 32. บน CommandHost เปิดไฟล์ `/home/ec2-user/change-resource-tags.sh`:

```
nano change-resource-tags.sh
```

- 33. ตรวจสอบเนื้อหาของสคริปต์:

```
#!/bin/bash
ids=$(aws ec2 describe-instances --filter "Name=tag:Project,Values=ERPSYSTEM" "Name=tag:Environment,Values=development" --query 'Reservations[*].Instances[*].InstanceId' --output text)
aws ec2 create-tags --resources $ids --tags 'Key=Version,Value=1.1'
```

- o สคริปต์นี้ใช้คำสั่ง `aws ec2 describe-instances` ก่อน เพื่อส่งรายการ instance IDs ของเครื่อง development ที่สังกัดโปรเจกต์ **ERPSYSTEM** เท่านั้น จากนั้นส่งค่าเหล่านั้นให้คำสั่ง `aws ec2 create-tags` ซึ่งจะสร้าง tag ใหม่หรือ (ในกรณีนี้) เขียนทับ tag ที่มีอยู่
- o สังเกตว่าคำสั่งแรกใช้ตัวเลือก **—output text** เพื่อจัดการผลลัพธ์เป็นข้อความแทน JSON การใช้แบบนี้แทน JSON กับผลลัพธ์ง่าย ๆ (ในกรณีนี้คือรายการ IDs) ทำให้จัดการผลลัพธ์และส่งต่อให้คำสั่งอื่นได้ง่ายขึ้น
- 34. ปิดโปรแกรมแก้ไข nano แล้วรันคำสั่งนี้จาก Linux command prompt:

```
./change-resource-tags.sh
```

- 35. เพื่อตรวจสอบว่าหมายเลขเวอร์ชันบน instances เหล่านั้นเพิ่มขึ้นแล้ว และ instances อื่นที่ไม่ใช่ development ในโปรเจกต์ **ERPSYSTEM** ไม่ได้รับผลกระทบ ให้คัดลอกและรันคำสั่งต่อไปนี้:

```
aws ec2 describe-instances --filter "Name=tag:Project,Values=ERPSYSTEM" --query 'Reservations[*].Instances[*].{ID:InstanceId, AZ:Placement.AvailabilityZone, Project:Tags[?Key==`Project`]| [0].Value, Environment:Tags[?Key==`Environment`]| [0].Value, Version:Tags[?Key==`Version`]| [0].Value}'
```

ส่วน 4 / 6

งานที่ 2: หยุดและเริ่มทรัพยากรตาม Tag

ในงานนี้ คุณจะใช้สคริปต์ที่เตรียมไว้ให้ หยุดและเริ่มชุด instances ที่ติด tag ว่าเป็น development instances

ตรวจสอบสคริปต์ Stopinator

- 36. บน Command Host Instance ให้ `cd` เข้าไดเรกทอรี **aws-tools** ในไดเรกทอรี home:

```
cd aws-tools
```

- 37. เปิดไฟล์ **stopinator.php** และตรวจสอบเนื้อหา:

```
nano stopinator.php
```

- o สคริปต์ **stopinator.php** เป็นสคริปต์อย่างง่ายที่ใช้ AWS SDK for PHP หยุดและเริ่ม instances ใหม่ตามชุด tags ซึ่งรองรับสถานการณ์อย่างการปิดเซิร์ฟเวอร์สภาพแวดล้อม development ตอนสิ้นวันและเปิดใหม่ตอนเช้า สคริปต์จะค้นหา instances ที่ตรงกับ tags ที่ระบุในทุก AWS region
- o สคริปต์รับอาร์กิวเมนต์ต่อไปนี้:
 - **-t:** ชุด tags ในรูปแบบ `name=value;name=value` สคริปต์แปลง tags เหล่านี้เป็นรูปแบบที่การเรียก AWS PHP `Ec2::DescribeInstance()` ต้องการ หากไม่ใส่พารามิเตอร์ทางเลือกนี้ สคริปต์จะระบุและปิด Amazon EC2 instances ทั้งหมดที่กำลังทำงานในบัญชี
 - **-s:** พารามิเตอร์แบบ Boolean ไม่ต้องใส่อาร์กิวเมนต์ เมื่อมีพารามิเตอร์นี้ instances ที่ระบุด้วย **-t** จะถูกเริ่ม (start) แทนที่จะถูกหยุด
- 38. ออกจากโปรแกรมแก้ไข nano

หยุดและเริ่มกระบวนการ Development ของ ERPPROJECT ใหม่

ในงานนี้ คุณจะใช้สคริปต์ **stopinator.php** ปิดและเปิดสภาพแวดล้อม development ของโปรเจกต์ **ERPSYSTEM** ใหม่

- 39. จาก Linux shell ให้รันสคริปต์ **stopinator.php**:

```
./stopinator.php -t"Project=ERPSYSTEM;Environment=development"
```

- o ผลลัพธ์ควรมีลักษณะดังนี้ แสดงว่าจะมีสอง instances ถูกหยุดใน AWS region ปัจจุบัน (ผลลัพธ์ของคุณจะต่างกันตาม region ที่แล็บทำงานอยู่)

```
Region is us-east-1
No instances to stop in region
Region is us-west-1
No instances to stop in region
Region is us-west-2
Found instance i-9552ba9f
Found instance i-d35fb7d9
Stopping all identified instances...
[...]
No instances to stop in region
Region is sa-east-1
No instances to stop in region
```

- 40. ที่เมนู **Services** เลือก **EC2**
- 41. ในแถบนำทาง เลือก **Instances**
- 42. ตรวจสอบว่าสอง instances กำลังหยุดหรือหยุดไปแล้ว

- 43. กลับไปที่ SSH session ของ Command Host และจาก Linux prompt เริ่ม instances ใหม่ด้วยคำสั่งต่อไปนี้:

```
./stopinator.php -t"Project=ERPSYSTEM;Environment=development" -s
```

- 44. กลับไปที่หน้าต่าง EC2 Management Console และตรวจสอบว่าสอง instances ที่ถูกปิดก่อนหน้านี้กำลังเริ่มทำงานใหม่

ส่วน 5 / 6

งานที่ 3: โจทย์ท้าทาย: terminate Instances ที่ไม่ปฏิบัติตามข้อกำหนด

ในโจทย์นี้ คุณจะถูกขอให้หาวิธี terminate instances ที่ไม่เป็นไปตามแนวทางการความปลอดภัยบางอย่าง

หมายเหตุ: หากคุณคุ้นเคยกับ AWS อยู่แล้ว เราแนะนำให้ลองทำโจทย์นี้ด้วยตัวเองก่อนอ่านเฉลยละเอียดในส่วนถัดไป เมื่อทำเสร็จแล้วให้ตรวจงานด้วยการทบทวนเฉลยละเอียด

คำอธิบายโจทย์ (Challenge Description)

สถานการณ์: บริษัทต้องการให้คุณสร้างกระบวนการอัตโนมัติที่จะ terminate instances ที่อาจเปิดช่องให้เกิดการละเมิดความปลอดภัยโดยอัตโนมัติ คุณได้ระบุรายการความเสี่ยงด้านความปลอดภัยแล้ว และกำลังตัดสินใจว่าจะทำอย่างไรมีประสิทธิภาพด้วยคำสั่ง AWS CLI หรือ PHP SDK for AWS

โจทย์: งานด้านความปลอดภัยงานแรกของของคุณง่าย: ค้นหา instances ทั้งหมดใน private subnet ที่ไม่ได้ติด tag **Environment** แล้ว terminate (หรือนโยบาย "tag-or-terminate" – ติด tag หรือถูก terminate)

คำใบ้ (Hints):

- หากคุณไม่ถนัด PHP หรือภาษาโปรแกรมที่คล้ายกัน (เช่น Python หรือ Ruby) ที่มี AWS SDK ให้ใช้ให้ลองใช้ชุดคำสั่ง AWS CLI ทำงานนี้
- การเรียก `AWS PHP Ec2::terminateInstances()` terminate instances ได้ คำสั่ง AWS CLI ที่เทียบเท่าคือ `aws ec2 terminate-instances`
- คุณใช้สคริปต์ **stopinator** จากส่วนที่ 2 เป็นแนวทางในการเขียนโค้ดได้

ภาพรวมเฉลย (Challenge Solution Overview)

มีหลายวิธีในการแก้โจทย์นี้ ด้วยโซลูชันแบบเขียนโปรแกรมหรือบรรทัดคำสั่งที่หลากหลาย วิธีแก้ทั่วไปประกอบด้วยขั้นตอนต่อไปนี้:

- ระบุ instances ทั้งหมดที่มี tag **Environment** อยู่ในขณะนี้
- เติบกับรายการ instances ทั้งหมดที่มีอยู่ และบันทึก instance IDs ของ instances ที่ไม่อยู่ในรายการที่ได้จากงานที่ 1
- ส่ง instance IDs ของ instances ที่ไม่ได้ติด tag ให้ AWS โดยใช้คำสั่ง `aws ec2 stop-instance` (AWS CLI – ตามต้นฉบับ) หรือการเรียก API `Ec2::terminateInstances()` (PHP)

เฉลยต่อไปนี้จะแสดงวิธีแก้โจทย์นี้ด้วยสคริปต์ PHP

งาน 3.1: ทบทวนสคริปต์ Tag-Or-Terminate

- 45. เปิดไฟล์ **terminate-instances.php** ด้วยโปรแกรมแก้ไข nano

```
nano terminate-instances.php
```

- 46. ตรวจสอบบล็อก **params** ของสคริปต์นี้ สังเกตว่ารับสองอาร์กิวเมนต์: region ปัจจุบันที่คุณทำงานอยู่ (**region**) และ ID ของ subnet (**subnetid**) โค้ดใช้อาร์กิวเมนต์ **subnetid** กำหนดว่าจะหา instances ที่ไม่ปฏิบัติตามข้อกำหนดที่ใด
- 47. ตรวจสอบบล็อกโค้ดแรก ที่ขึ้นต้นด้วยคอมเมนต์ # Obtain a list of all instances with the Environment tag set (หารายการ instances ทั้งหมดที่ตั้ง tag Environment ไว้)
 - o บล็อกโค้ดนี้ใช้เมธอด **describeInstances()** พร้อมตัวกรองเพื่อหา instances ทั้งหมดที่มี tag **Environment** ไม่ว่าค่าของ tag จะเป็นอะไร และเก็บ instance IDs ทั้งหมดที่พบไว้ใน hash table
- 48. ตรวจสอบบล็อกโค้ดที่สอง
 - o โค้ดนี้ตรวจสอบทุก instance ภายใน subnet ของคุณและเทียบกับรายการ instances ที่ติด tag **Environment** หาก instance ไม่อยู่ในรายการที่ติด tag ไว้ instance ID จะถูกเพิ่มในรายการ instances ที่จะ terminate
- 49. ตรวจสอบส่วนสุดท้ายของสคริปต์
 - o บรรทัดเหล่านี้ใช้รายการ instance IDs ที่ไม่ปฏิบัติตามข้อกำหนดเป็นอาร์กิวเมนต์ของเมธอด **terminateInstances**

ตั้งค่าสภาพแวดล้อมเพื่อทดสอบสคริปต์

ก่อนรันสคริปต์ คุณต้องแก้ instances สองสามตัวในแล็บให้ไม่มี tag **Environment** แล้ว

- 50. กลับไปที่ EC2 Management Console และสังเกต instances ที่ทำงานอยู่ในสภาพแวดล้อมแล็บ
- 51. เลือก instance หนึ่งตัวใน private subnet
- 52. ในแท็บ **Tags** ของ instance เลือก **Manage tags**
- 53. หา tag Environment แล้วเลือกไอคอน **remove** (ลบ)
- 54. เลือก **Save** ทำกระบวนการนี้ซ้ำกับ instance อีกหนึ่งตัวใน private subnet

รันสคริปต์

- 55. ใน EC2 Management Console เลือก instance หนึ่งตัวใน private subnet
- 56. ในแท็บ **Description** ของ instance หาช่อง **Availability zone** แล้วคัดลอกทั้งหมดยกเว้นตัวอักษรตัวสุดท้ายลงในไฟล์ข้อความ ในขั้นตอนถัดไปจะเรียกค่านี้ว่า **region**
- 57. หาช่อง **Subnet ID** แล้วคัดลอกค่าลงในไฟล์ข้อความ ในขั้นตอนถัดไปจะเรียกค่านี้ว่า **subnet-id**
- 58. กลับไปที่ SSH session แล้วรันสคริปต์ **terminate-instances.php** (แทน **<region>** ด้วย region ของคุณ และ **<subnet-id>** ด้วย subnet-id ของคุณ):

```
./terminate-instances.php -region <region> -subnetid <subnet-id>
```

- o คุณควรเห็นผลลัพธ์คล้ายกับ:

```
Checking i-dd3a90d1
Checking i-a4248ea8
Checking i-793a9075
Checking i-a9248ea5
Checking i-aa248ea6
Checking i-da3a90d6
Checking i-a13b91ad
Checking i-a23b91ae
Checking i-ab248ea7
Terminating instances...
Instances terminated.
```

ส่วน 6 / 6

แล็บเสร็จสิ้น (Lab Complete)

ยินดีด้วย! คุณทำแล็บเสร็จแล้ว

- 59. เลือก **End Lab** ที่ด้านบนของหน้านี้ แล้วเลือก **Yes** เพื่อยืนยันว่าต้องการจบแล็บ
 - o แผงจะปรากฏขึ้นพร้อมข้อความ "DELETE has been initiated... You may close this message box now." (เริ่มการลบแล้ว... คุณปิดกล่องข้อความนี้ได้)
- 60. เลือก **X** ที่มุมขวาบนเพื่อปิดแผง